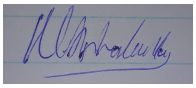
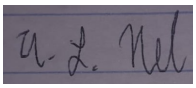
UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EDC 310

Digital Communications

Practical 1

Group 8

Name and Surname	Student Number	Signature	% Contribution
A. van Niekerk	19007583		33.33
N. Tshabalala	18171827		33.33
AS. Nel	19014432		33.33

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

September 22, 2021

Contents

Contents i

List of Figures i

List of Tables ii

1 Introduction 1

2 Questions 1
- 2.1 Question 1 . . . 1
  - 2.1.1 Statistics comparison . . . 1
  - 2.1.2 Probability distribution function . . . 2
- 2.2 Question 2 . . . 2
  - 2.2.1 Statistics comparison . . . 2
  - 2.2.2 Probability distribution function . . . 3
- 2.3 Question 3 . . . 4
  - 2.3.1 Bit Generation . . . 4
  - 2.3.2 Bit-to-symbol mapping . . . 4
  - 2.3.3 Gaussian noise . . . 7
  - 2.3.4 Symbol detection . . . 7
  - 2.3.5 Determining SER and BER . . . 7
  - 2.3.6 Plotting SER and BER as functions of SNR . . . 8
- 2.4 Discussion of results . . . 12

3 Conclusion 12

4 References 13

5 Appendix A: Python Code 14

List of Figures

1 Probability distribution function of the Uniform Random Number Generator 1
2 Probability distribution function of the Uniform Random Number Generator 2
3 Comparison of theoretical and implemented Gaussian Random Number Generators . . . 2
4 Probability distribution function of the Gaussian Random Number Generator 3
5 Theoretical Probability distribution function of a Gaussian distribution . 3
6 BPSk modulation constellation . . . 4
7 4QAM modulation constellation . . . 5
8 8PSK modulation constellation . . . 5
9 16QAM modulation constellation . . . 6
10 SER as a function of SNR for BPSK . . . 8
11 BER as a function of SNR for BPSK . . . 8
12 SER as a function of SNR for 4QAM . . . 9
13 BER as a function of SNR for 4QAM . . . 9

14 SER as a function of SNR for 8PSK 10
15 BER as a function of SNR for 8PSK 10
16 SER as a function of SNR for 16QAM 11
17 BER as a function of SNR for 16QAM 11

List of Tables

1 Introduction

The aim of practical 1 is to develop a simulation platform for a BPSK, 4QAM, 8PSK and 16QAM communication system transmitting information over an additive white Gaussian noise (AWGN) channel. Such a simulation platform will make use of a a uniform random number generator to generate random bits and a Gaussian random number generator to generate Gaussian noise. The platform will be developed in python and simulations will be run to evaluate the performance of the various subsections of the assignment.

2 Questions

2.1 Question 1

Question 1's assignment was to develop a uniform random number generator. The uniform random number generator must randomly generate numbers between 0 and 1 using the Wichmann-Hill algorithm [1]. The uniform random number generator must then be evaluated by comparing its statistics (μ, σ, σ^2) to theoretical statistics.

2.1.1 Statistics comparison

The performance of the Uniform Random Number Generator was evaluated by taking 200 samples from the coded uniform random number generator and compared to the statistics of the python random package's random number generator with the same number of samples.

```
For 200 samples
Theoretical statistics: (mu, sigma, sigma^2) =  ( 0.4791154417514448 0.2904660730323299 0.08437053958282283 )
Generated statistics: (mu, sigma, sigma^2) =  (0.49244567631315095, 0.3083348163459428, 0.09507035897108627)
```

Figure 1: Probability distribution function of the Uniform Random Number Generator

The values for μ, σ and σ^2 match those of the random() function of the random package. μ being 0.5, σ being 0.3 and σ^2 being 0.01.

2.1.2 Probability distribution function

The Probability distribution function was plotted using the pyplot package. The PDF was made use of 250000 samples.

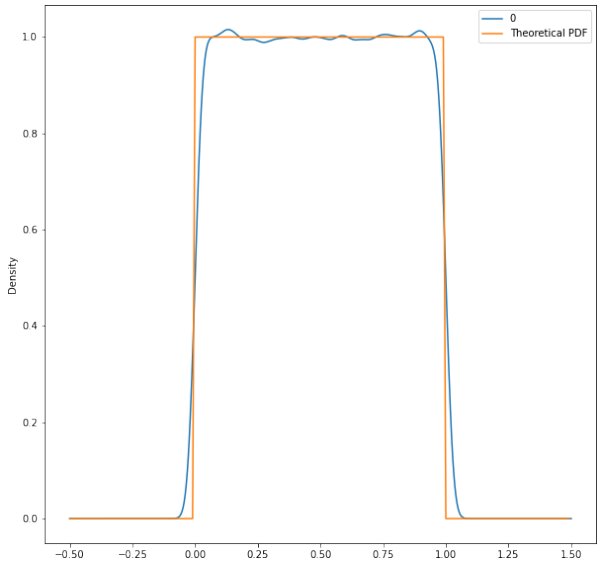


Figure 2: Probability distribution function of the Uniform Random Number Generator

2.2 Question 2

The second task involves developing a Gaussian RNG that generates random numbers that follow a distribution with $\mu = 0$ and $\sigma = 1$ – that is, the random numbers are normally distributed about a mean of zero with a standard deviation of one. The Gaussian RNG is developed using the Marsaglia-Bray algorithm [2]. The effectiveness of this RNG is then evaluated by comparing its statistics (μ, σ, σ^2) to that of an ideal Gaussian distribution.

2.2.1 Statistics comparison

In order to evaluate the performance of the Gaussian Random Number Generator, 10000000 samples were taken from the coded gaussian random number generator and compared to the statistics of the python random package's gaussian random number generator with the same number of samples.

```
With 1000000 samples and a bin size of 250
Theoretical statistics: (mu, sigma, sigma^2) =  ( -9.525259856654936e-05 1.0004577081552648 1.0009156258072849 )
Generated statistics: (mu, sigma, sigma^2) =  (3.867640824640813e-05, 1.00029598439959, 1.0005920564059447)
```

Figure 3: Comparison of theoretical and implemented Gaussian Random Number Generators

The values for μ,σ and σ^2 match those of the GRNG function of the random package. μ

being 0, and σ and σ^2 being 1.

2.2.2 Probability distribution function

The Probability distribution function was plotted using the "hist" function of the pyplot package. The PDF was made use of 1000000 samples using a bin size of 250.

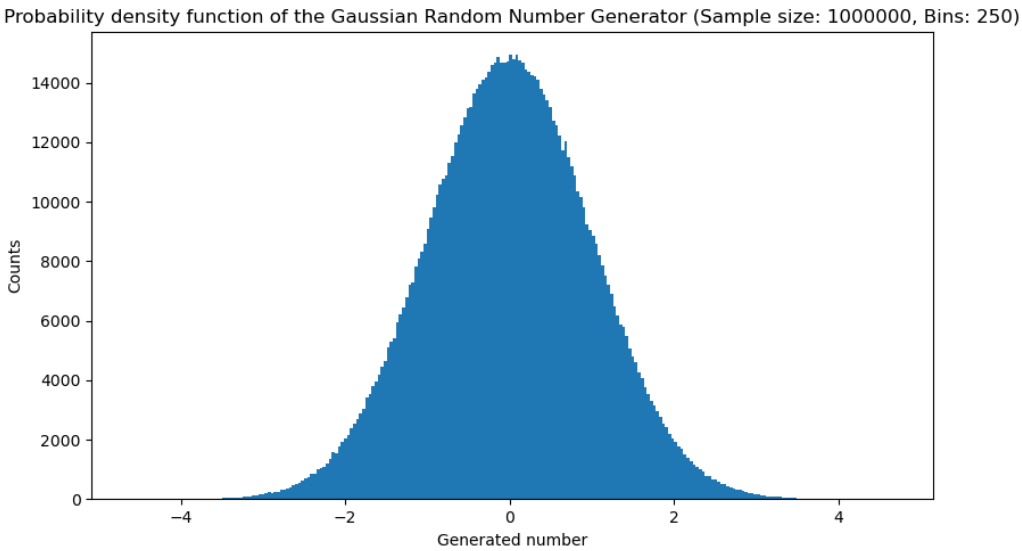


Figure 4: Probability distribution function of the Gaussian Random Number Generator

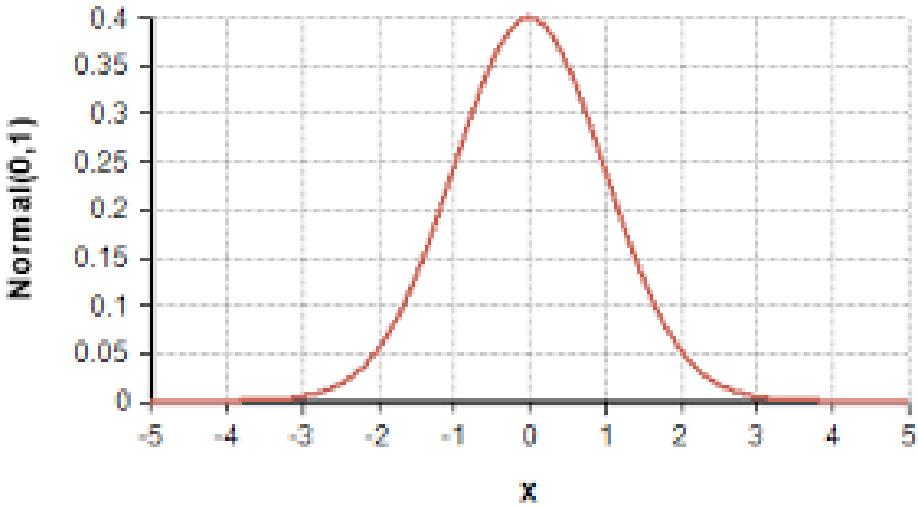


Figure 5: Theoretical Probability distribution function of a Gaussian distribution

The plot clearly shows the same shape in bell curve as the theoretical Gaussian distribution.

2.3 Question 3

Question 3 had students implement the actual simulation platform, making use of the uniform and gaussian random number generators from questions 1 and 2. The bit-error rate (BER) and symbol-error rate (SER) performance had to be evaluated in the range of [-4,12] dB.

2.3.1 Bit Generation

Random bits were generated using the uniform random number generator. Numbers greater than 0.5 represent a 1 and numbers less than 0.5 represent a 0.

2.3.2 Bit-to-symbol mapping

Bits were mapped to symbols using the respective modulation constellations.

For BPSK: S1 = 1, S2 = 0

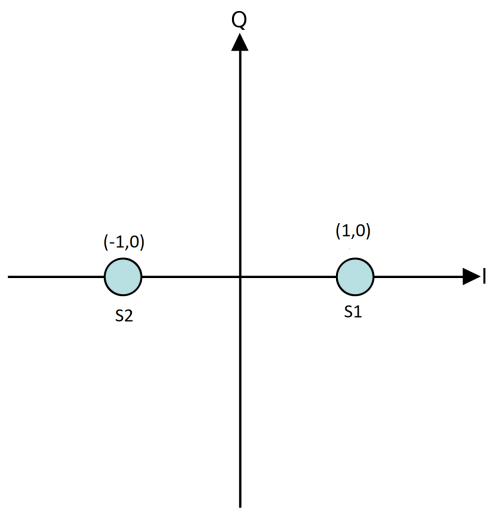


Figure 6: BPSk modulation constellation

For 4QAM: S1 = 00, S2 = 01, S3 = 10, S4 = 11

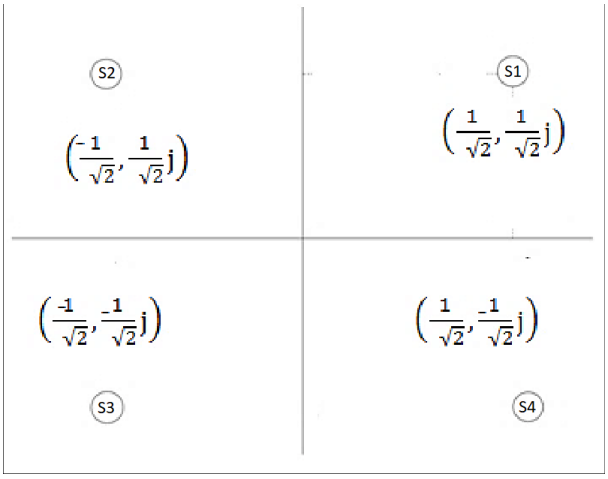


Figure 7: 4QAM modulation constellation

For 8PSK: S1 = 000, S2 = 001, S3 = 010, S4 = 011, S5 = 100, S6 = 101, S7 = 110, S8 = 111

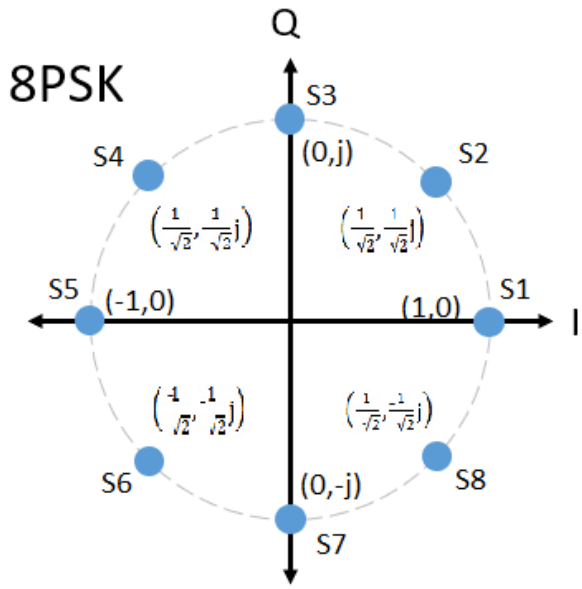


Figure 8: 8PSK modulation constellation

For 16qam: S1 = 1000, S2 = 1001, S3 = 1011, S4 = 1010, S5 = 1100, S6 = 1101, S7 = 1111, S8 = 1110, S9 = 0100, S10 = 0101, S11 = 0111, S12 = 0110, S13 = 0000, S14 = 0001, S15 = 0011, S16 = 0010

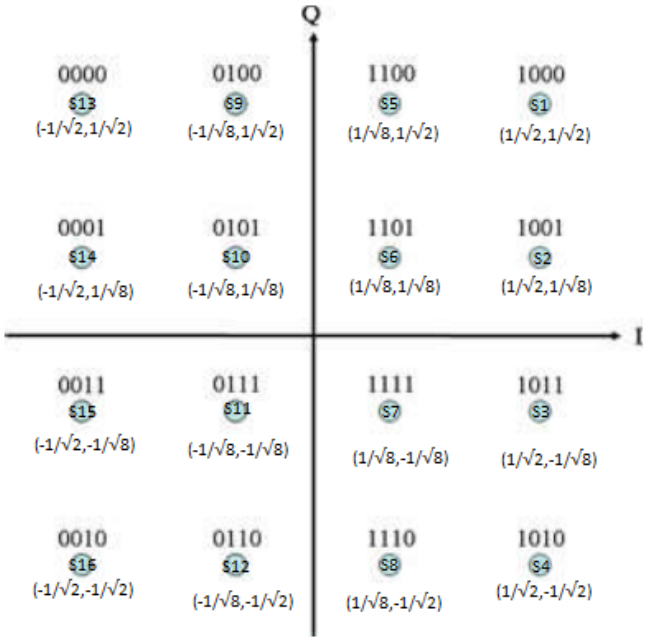


Figure 9: 16QAM modulation constellation

2.3.3 Gaussian noise

Noise was added to the generated bits using:

$$r_k = s_k + \sigma n_k,$$

where $n_k = (n_k^{(i)} + jn_k^{(q)})/\sqrt{2}$. $n_k^{(i)} and n_k^{(q)}$ are complex zero mean, unity variance, Gaussian random variables. Since

$$SNR = 10log\left(\frac{|a|}{2\sigma^2}\right) = 10log\left(\frac{1}{2\sigma^2}\right) = \frac{E_b}{N_0},$$

$$\sigma = \frac{1}{\sqrt{10^{\frac{E_b}{10N_0}} 2f_{bit}}}$$

where $f_{bit} = log_2(M)$ and M is the number of symbols in the modulation constellation.

2.3.4 Symbol detection

Received symbols are received by calculating the Euclidean distance between the received symbol and all other symbols in the constellation. The symbol with the smallest Euclidean distance is the most probable transmitted symbol.

$$\Delta^{(i)} = |r_t - s^{(i)}|^2$$

2.3.5 Determining SER and BER

Detected symbols are compared to transmitted symbols and symbol errors are counted. SER is determined by dividing the number of symbol errors by the number of transmitted symbols.

Detected symbols are converted back to bits and compared to the transmitted bits. The bit errors must then be counted. BER is determined by dividing the number of bit errors by the number of transmitted bits.

2.3.6 Plotting SER and BER as functions of SNR

The plots of the SER and BER as functions of SNR were done using the semilogy command in python.

Plot for BPSK:

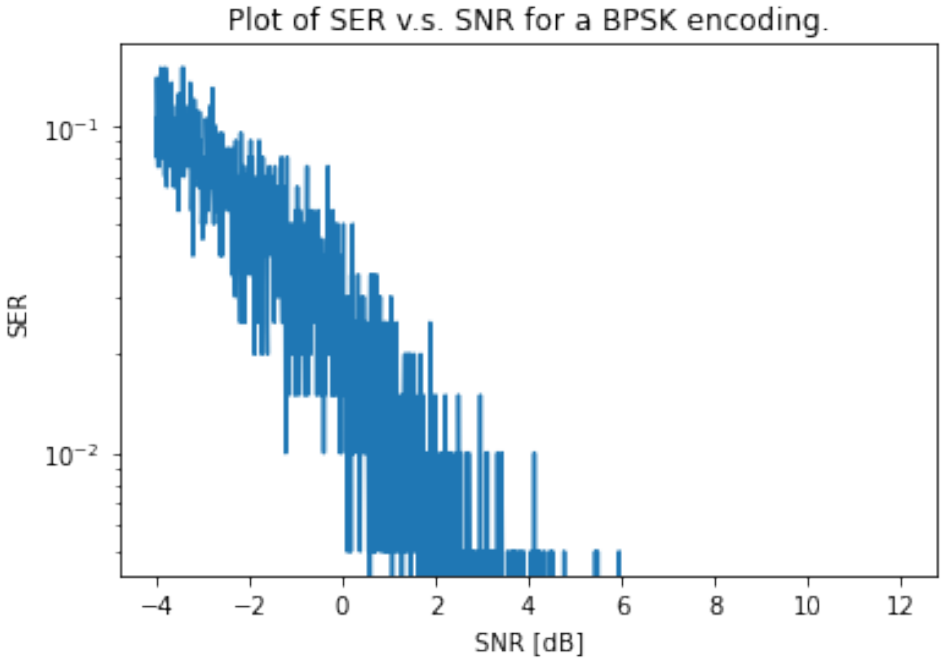


Figure 10: SER as a function of SNR for BPSK

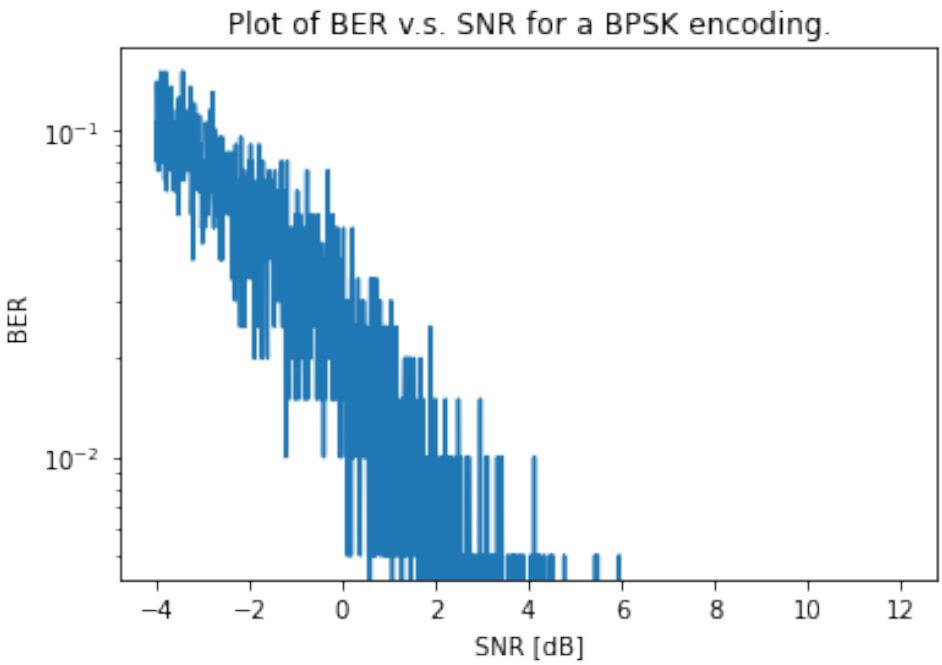


Figure 11: BER as a function of SNR for BPSK

Plot for 4QAM:

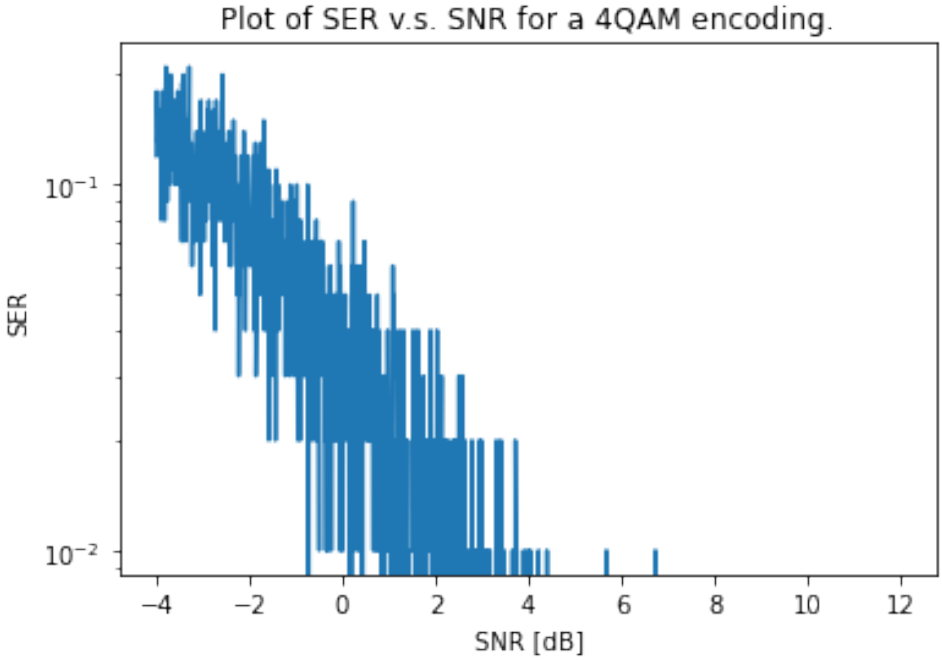


Figure 12: SER as a function of SNR for 4QAM

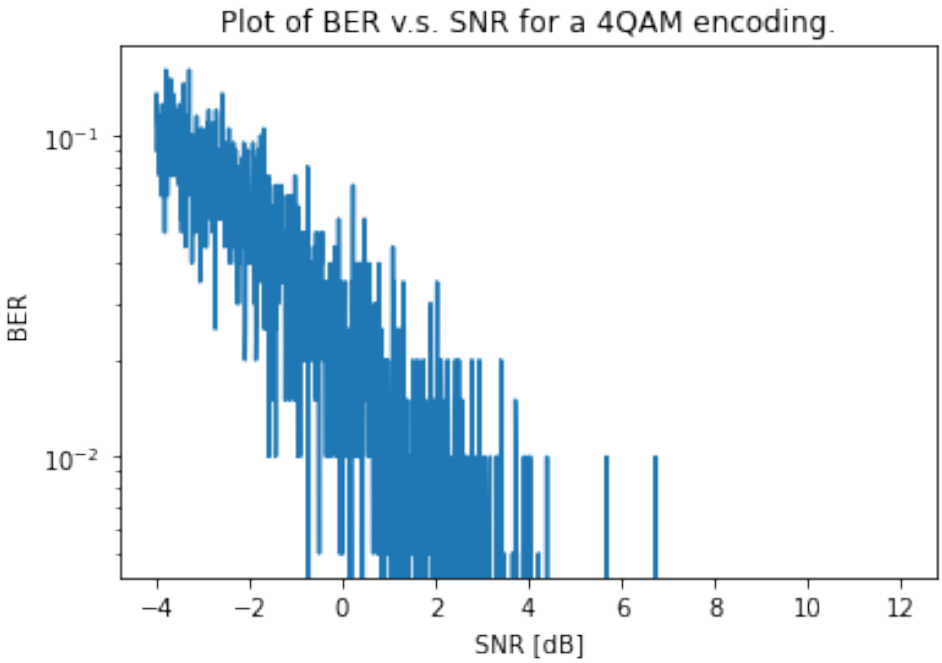


Figure 13: BER as a function of SNR for 4QAM

Plot for 8PSK:

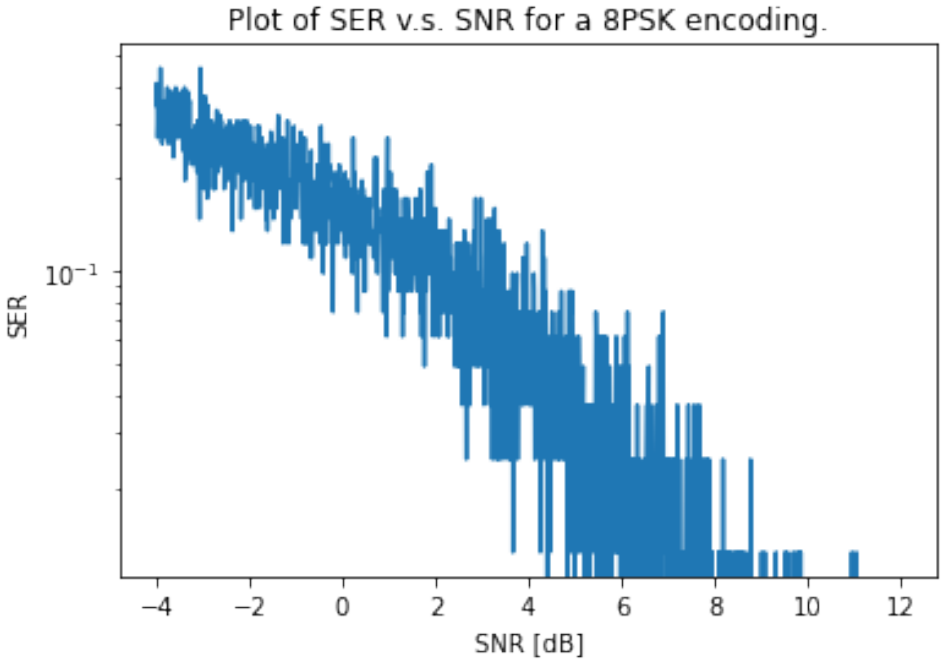


Figure 14: SER as a function of SNR for 8PSK

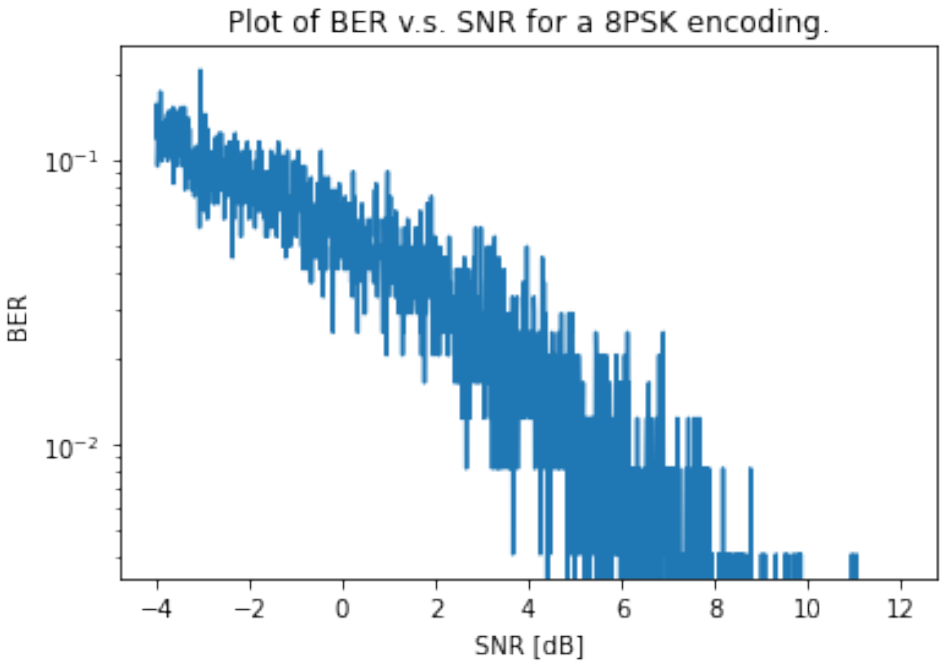


Figure 15: BER as a function of SNR for 8PSK

Plot for 16QAM:

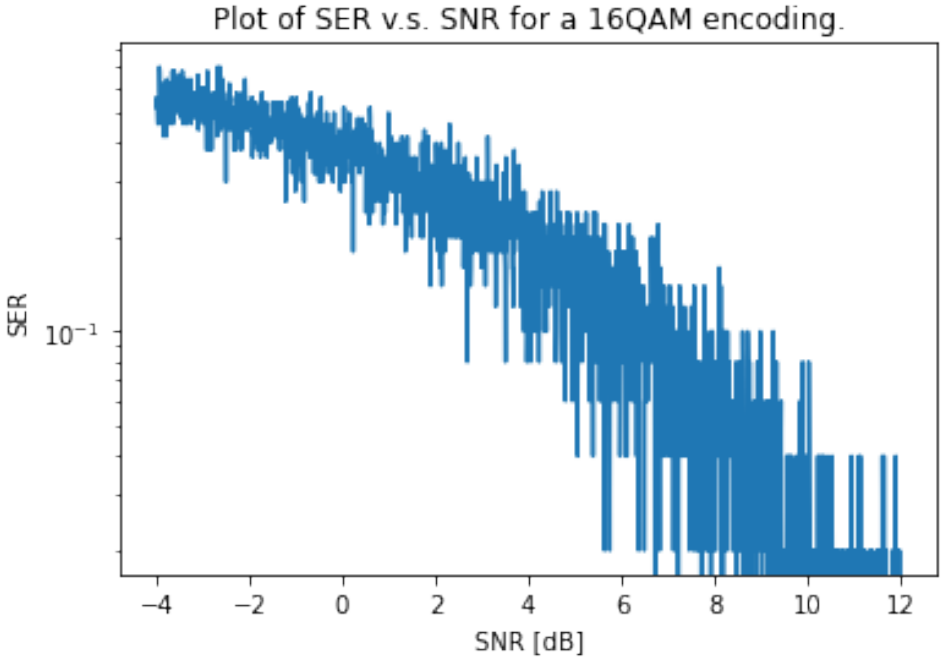


Figure 16: SER as a function of SNR for 16QAM

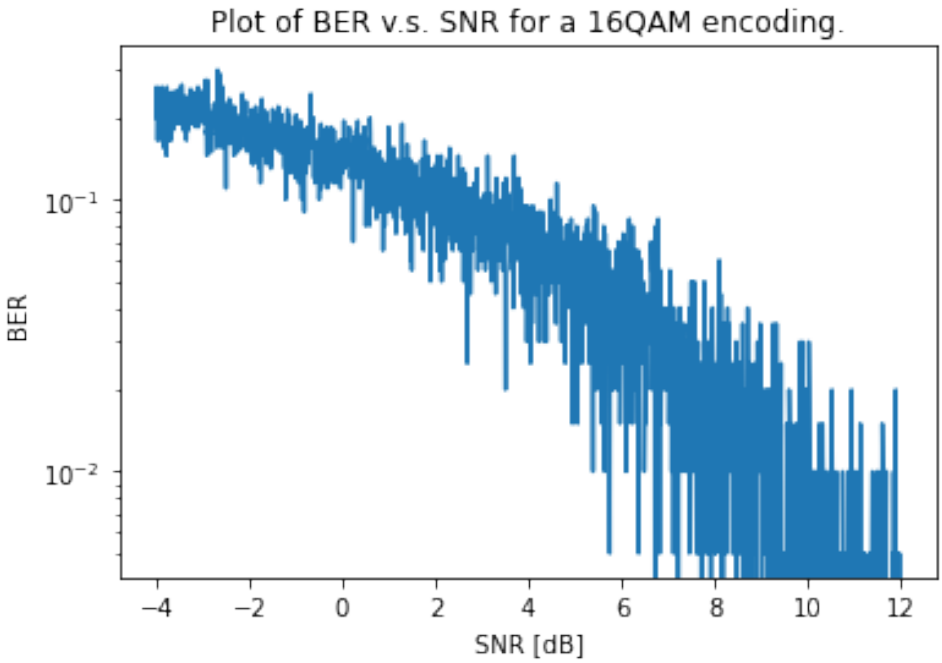


Figure 17: BER as a function of SNR for 16QAM

2.4 Discussion of results

The plots (Figures 10 to 17) provide a clear relationship between the signal-to-noise ratio and the bit/symbol-error rates. The greater the signal energy in comparison to the noise, the lower the error rate for both symbol and bit errors.

It is evident that the error rates become more pronounced for the same SNR when the number of symbols used in the modulation is increased – that is, comparing the error error rates associated with the BPSK modulation scheme, the simplest possible, with the other three modulation schemes investigated in this report, it is clear that BPSK modulation out performs the others in terms of bit and symbol accuracy for small energy signals with relatively high noise. This can be attributed to the relative complexities of modulation constellations. As the constellation becomes more complex, symbols become more densely populated, resulting in higher chances for errors to occur when noise is added.

Bit-errors are directly related to signal-errors, and an increase or decrease in the former has the same effect on the latter

3 Conclusion

Questions 1 and 2 were successfully implemented in order to code the simulation platform of question 3. Both the URNG and GRNG generated numbers that matched the theoretical statistics of their respective distribution models. These numbers could then be used to represent bits and add gaussian noise for transmitted symbols in the simulation platform.

When running the simulation, a clear relationship between the SER and SNR as well as between BER and SNR could be observed. The greater the signal to noise ration, the smaller the error rates for both signals and bits.

The practical was successfully implemented and allowed a better understanding of a BPSK, 4QAM, 8PSK and 16QAM communication system transmitting information over an additive white Gaussian noise channel.

4 References

[1] B. Wichmann and D. Hill, "Building a random number generator," *Byte*, pp. 127–128, 1987.

[2] G. Marsaglia and T. Bray, "A convenient method for generating normal variables," *SIAM Rev.*, vol. 6, pp. 260–264, 1964.

5 Appendix A: Python Code

Question 1

```
import doctest
import numpy as np
import random
import math
import time

%matplotlib inline
import matplotlib.pyplot as plt
import pandas as pd
import scipy.stats as stats

# declare global variables here, You may use global variables in functions by using
# Global seed variables
s1 = 10
s2 = 20
s3 = 30

def seed_unifrom(seed1:int, seed2:int, seed3:int)->None:
    """Advanced update of seed values for URNG
    :param seed1: the first seed value for WH Algorithm
    :param seed2: the second seed value for WH Algorithm
    :param seed3: the third seed value for WH Algorithm
    """
    global s1, s2, s3
    s1 = seed1
    s2 = seed2
    s3 = seed3
#end uniform()


def uniform()->float:
    """Wichmann-Hill algorithm
    Use global variables
    """
    # Write your Wichmann-Hill algorithm code here
    #Declaration of constant primes
    global s1, s2, s3
    constPrime1 = 40087
    constPrime2 = 50423
    constPrime3 = 61333

    #Declaration of arbitrary multipliers
    constMul1 = 347
    constMul2 = 346
```

```
    constMul3 = 348

    #Wichmann-Hill algorithm for updating the seeds
    s1 = (constMul1 * s1) % constPrime1
    s2 = (constMul2 * s2) % constPrime2
    s3 = (constMul3 * s3) % constPrime3

    #Restricting the result to [0,1)
    result = s1 / (constPrime1 * 1.0) + s2 / (constPrime2 * 1.0) + s3 /(constPrime3 *
    result = result - math.trunc(result)
    return result
#end uniform()

def calculate_uniform_stats(N:int)->tuple:
    """calculate the statistical parameters for the random sequence give
    :param N: the number of values to generate
    :return: tuple containing (mu, sigma, sigma_2) in that order
    """
    values = [uniform() for x in range(N)]

    mu = 0.0 #mean
    sigma = 0.0 #standard deviation
    sigma_2 = 0.0 # variance

    #Computing the mean (mu)
    for i in range(N):
        mu += values[i]
    mu = mu / (N * 1.0)

    #Computing the variance (sigma^2)
    for i in range(N):
        sigma_2 += (values[i] - mu)**2
    sigma_2 = sigma_2/(N - 1)

    #Computing the standard deviation (sigma)
    sigma = (sigma_2)**0.5
    return (mu, sigma, sigma_2)
#end calculate_uniform_stats()

print("(mu, sigma, sigma^2) = ", calculate_uniform_stats(200))

#Plot the PDF and compare it to the theoretical and stock RNGs

N = 250000# number of random numbers to generate

randArr = [uniform() for i in range(N)]
testArr = [np.random.uniform(0.0, 1.0) for i in range(N)]
```

```
#Plotting PDF of custom uniform RNG
pd.DataFrame(randArr).plot(kind = "density", figsize = (10, 10))
plt.title("PDF of the custom URNG")
#Plotting PDF of built in uniform RNG
pd.DataFrame(testArr).plot(kind = "density", figsize = (10, 10))

#Plotting PDF of theoretical uniform distribution
x_val = np.arange(-0.5, 1.5, 0.01)
y_val = []
plt.title("PDFs of the Python defined URNG and theoretical uniform distribution.")
for i in range(len(x_val)):
    if(0 <= x_val[i] and x_val[i] <= 1):
        y_val.append(1.0)
    else:
        y_val.append(0.0)

plt.plot(x_val, y_val, label = "Theoretical PDF")
plt.legend()
```

Question 2

```
import numpy as np
import random
import math
import time
import matplotlib.pyplot as plt

# Seed vaulues for initial randomnumber generation
global seed1
global seed2
seed1 = time.time()
seed2 = time.time()+2

# Advanced seed update function
def seed(seedvalue: int) -> None:
    """Lazy update of seed values for GRNG
        :param seedvalue: seed scale for GB Algorithm,
        Calls the normal seed function with
        s1 = seedvalue * 997
        s2 = seedvalue * 277
    """
    global seed1
    seed1 = seedvalue * 997
    global seed2
    seed2 = seedvalue * 277
    return

# Lazy seed update function
```

```python
def seed(s1: int, s2: int) -> None:
    """Advanced update of seed values for GRNG
    :param seed1: the first seed value for GH Algorithm
    :param seed2: the second seed value for GH Algorithm
    :param seed3: the third seed value for GH Algorithm
    """
    global seed1
    seed1 = s1
    global seed2
    seed2 = s2
    return

# Function to generate random gaussian numbers using the
# Marsaglia-Bray algorithm
def gauss():
    """Marsaglia-Bray algorithm
    Use global variables
    """
    while True:
        global seed1
        global seed2
        seed1 += 1
        seed2 += 1
        random.seed(seed1)
        v1 = random.random() * 2.0 - 1.0
        random.seed(seed2)
        v2 = random.random() * 2.0 - 1.0

        if ((v1 * v1 + v2 * v2 < 1.0) and (v1 * v1 + v2 * v2 > 0.0)):
            break

        seed1 += 1
        seed2 += 2
    s = v1 * v1 + v2 * v2

    x1 = v1 * np.sqrt(-2.0 * np.log(s) / s)
    x2 = v2 * np.sqrt(-2.0 * np.log(s) / s)

    return random.choice([x1,x2])

# Creates an array of gaussian random numbers and returns the mean,
# standard deviation and variance of the number set
def calculate_gauss_stats(N: int) -> tuple:
    """calculate the statistical parameters for the random sequence give
    :param N: the number of values to generate
    :return: tuple containing (mu, sigma, sigma_2) in that order
    """
    values = [gauss() for x in range(N)]
```

```
    mu = 0
    sigma = 1

    for i in range(len(values)):
        values[i] = values[i]*sigma
    for i in range(len(values)):
        values[i] = values[i] + mu
    return  (np.mean(values), np.std(values), np.var(values))

# Theoretical and calculated statistics comparison
"""
    1000000 samples are generated using the gauss() function and then
    again using the gaussian random number generator of the
    random package.
    The mean, standard deviation and variance is calulated for both and
    displayed for comparison purposes
"""
x = [random.gauss(0,1) for x in range(1000000)]

print("With 1000000 samples and a bin size of 250")
print("Theoretical statistics: (mu, sigma, sigma^2) = ",
      "(",np.mean(x), np.std(x), np.var(x),")")


# PDF plot
"""
    nums variable is populated with 1000000 numbers using the GRNG
    and plotted as a histogram. The two seed values are updated in
    order to
    ensure that the same number isn't continuously generated.
"""
nums = []
for i in range(1000000):
    seed1 += 1
    seed2 += 1
    nums.append(gauss())
plt.hist(nums, bins = 250)
plt.title("Probability density function of the Gaussian Random Number "
          "Generator (Sample size: 1000000, Bins: 250)")
plt.xlabel("Generated number")
plt.ylabel("Counts")
plt.show()

print("Generated statistics: (mu, sigma, sigma^2) = ",
      calculate_gauss_stats(1000000))
```

Question 3

```python
import numpy as np
import random
import math
import time
import matplotlib.pyplot as plt


# Global seed variables
s1 = 10
s2 = 20
s3 = 30

def seed_unifrom(seed1:int, seed2:int, seed3:int)->None:
    """Advanced update of seed values for URNG
    :param seed1: the first seed value for WH Algorithm
    :param seed2: the second seed value for WH Algorithm
    :param seed3: the third seed value for WH Algorithm
    """
    global s1, s2, s3
    s1 = seed1
    s2 = seed2
    s3 = seed3
#end uniform()


def uniform()->float:
    """Wichmann-Hill algorithm
    Use global variables
    """
    # Write your Wichmann-Hill algorithm code here
    #Declaration of constant primes
    global s1, s2, s3
    constPrime1 = 40087
    constPrime2 = 50423
    constPrime3 = 61333

    #Declaration of arbitrary multipliers
    constMul1 = 347
    constMul2 = 346
    constMul3 = 348

    #Wichmann-Hill algorithm for updating the seeds
    s1 = (constMul1 * s1) % constPrime1
    s2 = (constMul2 * s2) % constPrime2
    s3 = (constMul3 * s3) % constPrime3

    #Restricting the result to [0,1)
    result = s1 / (constPrime1 * 1.0) + s2 / (constPrime2 * 1.0) + s3 /(constPrime3 *
```

```
    result = result - math.trunc(result)
    return result
#end uniform()

global seed1
global seed2
seed1 = time.time()
seed2 = time.time()+2

def seed_gauss(s1:int, s2:int)->None:
    """Advanced update of seed values for GRNG
    :param seed1: the first seed value for GH Algorithm
    :param seed2: the second seed value for GH Algorithm
    """
    global seed1
    seed1 = s1
    global seed2
    seed2 = s2
    return

def gauss()->float:
    """Marsaglia-Bray algorithm
    Use global variables
    """
    while True:
        global seed1
        global seed2
        seed1 += 1
        seed2 += 1
        random.seed(seed1)
        v1 = random.random() * 2.0 - 1.0
        random.seed(seed2)
        v2 = random.random() * 2.0 - 1.0

        if ((v1 * v1 + v2 * v2 < 1.0) and (v1 * v1 + v2 * v2 > 0.0)):
            break

        seed1 += 1
        seed2 += 2
    s = v1 * v1 + v2 * v2

    x1 = v1 * np.sqrt(-2.0 * np.log(s) / s)
    x2 = v2 * np.sqrt(-2.0 * np.log(s) / s)

    return random.choice([x1,x2])

def generate_N_bits(N)->tuple:
    # Use your uniform random number generator to generate random bits. A value larg
```

```
    # than 0.5 is a 1 bit; a value less than or equal to 0.5 is a 0 bit. This will
    """
        :param N: the number of values to generate
        :templis: temporary list to store values and later convert to tuple
        :return Tuplle containing random generated bits
    """
    templist = []
    for i in range(N):
        templist.append(int(np.round(uniform())))
    return tuple(templist) #convert list bit_stream into tuple and return
#end generate_N_bits()

def bits_to_symbols(bits:tuple, constellations='bpsk')->tuple:
    # Map the bits to symbols using the respective modulation constellations
    """
        :param bits: tuple of bits to be converted to symbols
        :param constellations: Constellation used for symbol conversion
        :temp: temporary list to store values and later convert to tuple
        :return Tuplle containing the converted symbols
    """
    temp = []
    message = ()
    if(constellations == 'bpsk'):
        for i in range(len(bits)):
            if(bits[i] == 1):#s1
                temp.append(1)
            elif(bits[i] == 0):#s2
                temp.append(-1)
    #end of constellations == 'bpsk'

    elif(constellations == '4qam'):
        if(len(bits) % 2 == 0): #if the number of is even
            i = 0
            while(i < len(bits) - 1):
                if(bits[i] == 0 and bits[i + 1] == 0):
                    temp.append(round(1/2**0.5,4) + round(1/2**0.5,4)*1j) #s1

                elif(bits[i] == 0 and bits[i + 1] == 1):
                    temp.append(round(-1/2**0.5,4) + round(1/2**0.5,4)*1j) #s2

                elif(bits[i] == 1 and bits[i + 1] == 1):
                    temp.append(round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j) #s3

                elif(bits[i] == 1 and bits[i + 1] == 0):
                    temp.append(round(1/2**0.5,4) + round(-1/2**0.5,4)*1j) #s4

                i += 2#post-increment index
```

```
    #else: #for the case when there's an odd number of bits in the stream
#end of constellations == '4qam'

elif(constellations == '8psk'):
    if(len(bits) % 3 == 0):
        i = 0
        while(i < len(bits) - 2):
            if(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 1): #s1
                temp.append(1)

            elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 0): #s2
                temp.append(round(1/2**0.5,4) + round(1/2**0.5,4)*1j)

            elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 0): #s3
                temp.append(1j)

            elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 1): #s4
                temp.append(round(-1/2**0.5,4) + round(1/2**0.5,4)*1j)

            elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 1): #s5
                temp.append(-1)

            elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 0): #s6
                temp.append(round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j)

            elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 0): #s7
                temp.append(-1j)

            elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 1): #s8
                temp.append(round(1/2**0.5,4) + round(-1/2**0.5,4)*1j)

            i += 3 #post-increment index

    #else: #for the case when the number of bits in the stream isn't a multiple
#end of constellations == '8psk'

elif(constellations == '16qam'):
    if(len(bits) % 4 == 0):
        i = 0
        while(i < len(bits) - 3):
            if(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[i
                temp.append(round(1/2**0.5,4) + round(1/2**0.5,4)*1j)

            elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[
                temp.append(round(1/2**0.5,4) + round(1/8**0.5,4)*1j)

            elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[
                temp.append(round(1/2**0.5,4) + round(-1/8**0.5,4)*1j)
```

```
                elif(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[
                    temp.append(round(1/2**0.5,4) + round(-1/2**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[
                    temp.append(round(1/8**0.5,4) + round(1/2**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[
                    temp.append(round(1/8**0.5,4) + round(1/8**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[
                    temp.append(round(1/8**0.5,4) + round(-1/8**0.5,4)*1j)

                elif(bits[i] == 1 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[
                    temp.append(round(1/8**0.5,4) + round(-1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[
                    temp.append(round(-1/8**0.5,4) + round(1/2**0.5,4)*1j )

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 0 and bits[
                    temp.append(round(-1/8**0.5,4) + round(1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[
                    temp.append(round(-1/8**0.5,4) + round(-1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 1 and bits[i + 2] == 1 and bits[
                    temp.append(round(-1/8**0.5,4) + round(-1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[
                    temp.append(round(-1/2**0.5,4) + round(1/2**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 0 and bits[
                    temp.append(round(-1/2**0.5,4) + round(1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[
                    temp.append(round(-1/2**0.5,4) + round(-1/8**0.5,4)*1j)

                elif(bits[i] == 0 and bits[i + 1] == 0 and bits[i + 2] == 1 and bits[
                    temp.append(round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j)

                i += 4
        #else: #For the case when the number of transmitted bits isn't a multiple o
    #end constellation == '16qam'
    message = tuple(temp)
    return message
#end bits_to_symbols()

def get_fbits(constellation='bpsk'):
```

```
    """
        This function returns the number of bits per
        symbol for a constellation.
    """
    if(constellation=='bpsk'):
        return 1
    elif(constellation == '4qam'):
        return 2
    elif(constellation == '8psk'):
        return 3
    elif(constellation == '16qam'):
        return 4
#end get_fbits()

def sigma(snr:float, f_bits:int):
    """
        This function computes and returns the sigma
        value used in the computation of symbol noise
        based on the number of bits per symbol in an
        encoding scheme and the signal to noise ratio.
    """
    return (1/np.sqrt(10**(snr/10)*2*f_bits))
#end sigma()

def add_noise(symbols:tuple, snr:float, f_bits:int)->tuple:
    """
        This function computes and adds noise to the
        symbols tuple passed to it based on the SNR and
        f_bits passed in its parameters.
    """
    temp = []
    sigma_x = sigma(snr, f_bits)
    for i in range(len(symbols)):
        random_gaus = gauss()
        to_add = sigma_x*random_gaus
        temp_val = symbols[i]
        temp_val = temp_val + complex(to_add, to_add)/math.sqrt(2)
        temp.append(temp_val)

    return tuple(temp)
#end add_noise()

def euclidean_distance(symbol, constellation='bpsk')->dict:
    # return the distances between the given symbol and every symbol in the given co
    # return your result as a dictionary, with the symbol as the key to its distance
    # eg {<symbol_1>: distance_1, ... , <symbol_n>: distance_1 }
    # >>> eculidean_distance( 0.34, 'bpsk')
    # {1: 0.66 , -1: 1.34}
```

```
if(constellation=='bpsk'):
    s1 = round(math.sqrt(abs(symbol - 1)**2),2)
    s2 = round(math.sqrt(abs(symbol + 1)**2),2)
    dictionarry = {1:s1, -1:s2}
    return dictionarry
elif(constellation == '4qam'):
    s1 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/2**0.5))**2),2)
    s2 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/2**0.5))**2),2)
    s3 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/2**0.5))**2),2)
    s4 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/2**0.5))**2),2)

    dictionarry = {round(1/2**0.5,4) + round(1/2**0.5,4)*1j: s1, round(-1/2**0.5,
    + round(1/2**0.5,4)*1j: s2, round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j:s3,
    round(1/2**0.5,4) + round(-1/2**0.5,4)*1j:s4}
    return dictionarry

elif(constellation == '8psk'):
    s1 = round(math.sqrt(abs(symbol - 1)**2),2)
    s2 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/2**0.5))**2),2)
    s3 = round(math.sqrt(abs(symbol - complex(0, 1))**2),2)
    s4 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/2**0.5))**2),2)
    s5 = round(math.sqrt(abs(symbol - (-1))**2),2)
    s6 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/2**0.5))**2),2)
    s7 = round(math.sqrt(abs(symbol - (complex(0, -1)))**2),2)
    s8 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/2**0.5))**2),2)

    dictionarry = {1:s1, round(1/2**0.5,4) + round(1/2**0.5,4)*1j : s2, 1j : s3,
    round(-1/2**0.5,4) + round(1/2**0.5,4)*1j : s4, -1 : s5, round(-1/2**0.5,4) +
    round(-1/2**0.5,4)*1j : s6, -1j: s7, round(1/2**0.5,4) + round(-1/2**0.5,4)*1
    return dictionarry

elif(constellation == '16qam'):
    s1 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/2**0.5))**2),2)
    s2 = round(math.sqrt(abs(symbol - complex(1/2**0.5, 1/8**0.5))**2),2)
    s3 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/8**0.5))**2),2)
    s4 = round(math.sqrt(abs(symbol - complex(1/2**0.5, -1/2**0.5))**2),2)
    s5 = round(math.sqrt(abs(symbol - complex(1/8**0.5, 1/2**0.5))**2),2)
    s6 = round(math.sqrt(abs(symbol - complex(1/8**0.5, 1/8**0.5))**2),2)
    s7 = round(math.sqrt(abs(symbol - complex(1/8**0.5, -1/8**0.5))**2),2)
    s8 = round(math.sqrt(abs(symbol - complex(1/8**0.5, -1/2**0.5))**2),2)
    s9 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, 1/2**0.5))**2),2)
    s10 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, 1/8**0.5))**2),2)
    s11 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, -1/8**0.5))**2),2)
    s12 = round(math.sqrt(abs(symbol - complex(-1/8**0.5, -1/2**0.5))**2),2)
    s13 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/2**0.5))**2),2)
    s14 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, 1/8**0.5))**2),2)
    s15 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/8**0.5))**2),2)
    s16 = round(math.sqrt(abs(symbol - complex(-1/2**0.5, -1/2**0.5))**2),2)
```

```
        dictionarry = {round(1/2**0.5,4) + round(1/2**0.5,4)*1j : s1,
        round(1/2**0.5,4) + round(1/8**0.5,4)*1j :s2,
        round(1/2**0.5,4) + round(-1/8**0.5,4)*1j :s3,
        round(1/2**0.5,4) + round(-1/2**0.5,4)*1j :s4,
        round(1/8**0.5,4) + round(1/2**0.5,4)*1j :s5,
        round(1/8**0.5,4) + round(1/8**0.5,4)*1j: s6,
        round(1/8**0.5,4) + round(-1/8**0.5,4)*1j:s7,
        round(1/8**0.5,4) + round(-1/2**0.5,4)*1j :s8,
        round(-1/8**0.5,4) + round(1/2**0.5,4)*1j :s9,
        round(-1/8**0.5,4) + round(1/8**0.5,4)*1j :s10,
        round(-1/8**0.5,4) + round(-1/8**0.5,4)*1j :s11,
        round(-1/8**0.5,4) + round(-1/2**0.5,4)*1j :s12,
        round(-1/2**0.5,4) + round(1/2**0.5,4)*1j :s13,
        round(-1/2**0.5,4) + round(1/8**0.5,4)*1j :s14,
        round(-1/2**0.5,4) + round(-1/8**0.5,4)*1j :s15,
        round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j :s16}
        return dictionarry
#end euclidean_distance()

def symbol_detection(symbols:tuple, constellation='bpsk') -> tuple:
    # return a tuple of the symbols based on the eculidean_distances results
    temp = []

    for i in range(len(symbols)):
        temp_key = ''
        value = 999999999

        temp_dic = euclidean_distance(symbols[i], constellation)

        for key in temp_dic:
            if temp_dic[key] <=  value:
                value = temp_dic[key]

        for key in temp_dic:
            if temp_dic[key] == value:
                temp.append(key)
                break

    return tuple(temp)
#end symbol_detection()

def bit_detection(symbols:tuple, constellation='bpsk') -> tuple:
    """return a tuple of the bits based on the results of the symbols
    >>> bit_detection((1, -1, 1, -1), 'bpsk')
    (1, 0, 1, 0)
    >>> bit_detection((1, 1, 1, -1), 'bpsk')
    (1, 1, 1, 0)
```

```
"""
temp = []
tempsymbols = symbol_detection(symbols, constellation)
if(constellation == 'bpsk'):
    for i in range(len(tempsymbols)):
        if(tempsymbols[i] == 1):
            temp.append(1)

        elif(tempsymbols[i] == -1):
            temp.append(0)
#end of constellations == 'bpsk'

elif(constellation == '4qam'):
    for i in range(len(tempsymbols)):
        if(tempsymbols[i] == round(1/2**0.5,4) + round(1/2**0.5,4)*1j):
            temp.append(0)
            temp.append(0)

        elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/2**0.5,4)*1j):
            temp.append(0)
            temp.append(1)

        elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j):
            temp.append(1)
            temp.append(1)

        elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/2**0.5,4)*1j):
            temp.append(1)
            temp.append(0)

#end of constellations == '4qam'

elif(constellation == '8psk'):
    for i in range(len(tempsymbols)):
        if(tempsymbols[i] == 1):
            temp.append(1)
            temp.append(1)
            temp.append(1)

        elif(tempsymbols[i] == round(1/2**0.5,4) + round(1/2**0.5,4)*1j): #s2
            temp.append(1)
            temp.append(1)
            temp.append(0)

        elif(tempsymbols[i] == 1j): #s3
            temp.append(0)
            temp.append(1)
            temp.append(0)
```

```
        elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/2**0.5,4)*1j): #s4
            temp.append(0)
            temp.append(1)
            temp.append(1)

        elif(tempsymbols[i] == -1): #s5
            temp.append(0)
            temp.append(0)
            temp.append(1)

        elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j): #s6
            temp.append(0)
            temp.append(0)
            temp.append(0)

        elif(tempsymbols[i] == -1j): #s7
            temp.append(1)
            temp.append(0)
            temp.append(0)

        elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/2**0.5,4)*1j): #s8
            temp.append(1)
            temp.append(0)
            temp.append(1)

    #end of constellations == '8psk'

    elif(constellation == '16qam'):
        for i in range(len(tempsymbols)):
            if(tempsymbols[i] == round(1/2**0.5,4) + round(1/2**0.5,4)*1j): #(1000)S1
            temp.append(1)
            temp.append(0)
            temp.append(0)
            temp.append(0)

        elif(tempsymbols[i] == round(1/2**0.5,4) + round(1/8**0.5,4)*1j): #(1001)
            temp.append(1)
            temp.append(0)
            temp.append(0)
            temp.append(1)

        elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/8**0.5,4)*1j): #(1011
            temp.append(1)
            temp.append(0)
            temp.append(1)
            temp.append(1)
```

```
        elif(tempsymbols[i] == round(1/2**0.5,4) + round(-1/2**0.5,4)*1j): #(1010
            temp.append(1)
            temp.append(0)
            temp.append(1)
            temp.append(0)

        elif(tempsymbols[i] == round(1/8**0.5,4) + round(1/2**0.5,4)*1j): #(1100)
            temp.append(1)
            temp.append(1)
            temp.append(0)
            temp.append(0)

        elif(tempsymbols[i] == round(1/8**0.5,4) + round(1/8**0.5,4)*1j): #(1101)
            temp.append(1)
            temp.append(1)
            temp.append(0)
            temp.append(1)

        elif(tempsymbols[i] == round(1/8**0.5,4) + round(-1/8**0.5,4)*1j): #(1111
            temp.append(1)
            temp.append(1)
            temp.append(1)
            temp.append(1)

        elif(tempsymbols[i] == round(1/8**0.5,4) + round(-1/2**0.5,4)*1j): #(1110
            temp.append(1)
            temp.append(1)
            temp.append(1)
            temp.append(0)

        elif(tempsymbols[i] == round(-1/8**0.5,4) + round(1/2**0.5,4)*1j ): #(010
            temp.append(0)
            temp.append(1)
            temp.append(0)
            temp.append(0)

        elif(tempsymbols[i] == round(-1/8**0.5,4) + round(1/8**0.5,4)*1j): #(0101
            temp.append(0)
            temp.append(1)
            temp.append(0)
            temp.append(1)

        elif(tempsymbols[i] == round(-1/8**0.5,4) + round(-1/8**0.5,4)*1j): #(011
            temp.append(0)
            temp.append(1)
            temp.append(1)
            temp.append(1)
```

```
            elif(tempsymbols[i] == round(-1/8**0.5,4) + round(-1/2**0.5,4)*1j): #(011
                temp.append(0)
                temp.append(1)
                temp.append(1)
                temp.append(0)

            elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/2**0.5,4)*1j): #(0000
                temp.append(0)
                temp.append(0)
                temp.append(0)
                temp.append(0)

            elif(tempsymbols[i] == round(-1/2**0.5,4) + round(1/8**0.5,4)*1j): #(0001
                temp.append(0)
                temp.append(0)
                temp.append(0)
                temp.append(1)

            elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/8**0.5,4)*1j): #(001
                temp.append(0)
                temp.append(0)
                temp.append(1)
                temp.append(1)

            elif(tempsymbols[i] == round(-1/2**0.5,4) + round(-1/2**0.5,4)*1j): #(001
                temp.append(0)
                temp.append(0)
                temp.append(1)
                temp.append(0)

    return tuple(temp)
#end symbol_detection()

def calculate_SER(transmitted:tuple, received:tuple)->float:
    """
        This function determines the symbol error rate bteween the transmitted and r
        symbols passed into the function as Tupples
    """
    counter = 0
    for i in range(len(transmitted)):
        if(transmitted[i] != received[i]):
            counter += 1

    total_transmitted = len(transmitted)
    return counter/total_transmitted
#end calculate_SER()

def calculate_BER(transmitted:tuple, received:tuple)->float:
```

```
    """
        This function determines the bit error rate bteween the transmitted and rec
        bits passed into the function as Tupples
    """
    counter = 0
    for i in range(len(transmitted)):
        if(transmitted[i] != received[i]):
            counter += 1

    total_transmitted = len(transmitted)
    return counter/total_transmitted
#end calculate_BER()

'''
The relevant libraries have be imported above.
import matplotlib.pyplot as plt
import numpy as np
'''
constellations = ['bpsk', '4qam', '8psk', '16qam']
bits_per_constellation = [200, 200, 243, 200] # number of bits to be transmitted for
SNR_array = np.arange(-4, 12, 0.01) #signal to noise ratio array

for i in range(len(constellations)): # For each constellation compute the following:
    SER_array = [] #symbol error rate array
    BER_array = [] #bit error rate array

    transmitted_bits = generate_N_bits(bits_per_constellation[i]) #generate bits to b
    transmitted_symbols = bits_to_symbols(transmitted_bits, constellations[i]) #modul

    for j in range(len(SNR_array)): # For each SNR applied to a contellation compute
        noisy_symbols = add_noise(transmitted_symbols, SNR_array[j], get_fbits(conste
        recieved_symbols = symbol_detection(noisy_symbols, constellations[i])
        recieved_bits = bit_detection(recieved_symbols, constellations[i])

        SER_array.append(calculate_SER(transmitted_symbols, recieved_symbols))
        BER_array.append(calculate_BER(transmitted_bits, recieved_bits))

    plt.figure(2 * i + 1)
    plt.semilogy(SNR_array, SER_array)
    plt.title("Plot of SER v.s. SNR for a " + constellations[i].upper() +" encoding."
    plt.ylabel("SER")
    plt.xlabel("SNR [dB]")

    plt.figure(2 * i + 2)
    plt.semilogy(SNR_array, BER_array)
    plt.title("Plot of BER v.s. SNR for a " + constellations[i].upper() +" encoding."
    plt.ylabel("BER")
    plt.xlabel("SNR [dB]")
```

```
plt.show()
```